

Programming Techniques for Scientific Simulations I (401-2813-00L)

News

- Lecture begins on Thursday, September 18, 14:15. (08.09.2025)

General information

- **Team:** Roger Käppeli, Dr. Federico Gatti, Alexander Pietak, Jonas Bachmann, Lionel Brodbeck, and Maximilian Kryukov
- **Lecture/Exercises:** Thursday 14:15 - 17:45, HG E 1.1 »
- **Online Q&A:** Monday, 17:00 - 18:00. Registration is required to attend the session. (If you are unable to join, please cancel your registration in advance.) Link to the Zoom Meeting ID can be found here. (You need your ETH login for web applications (LDAP).)
- **Exercise submission:** Until Monday night; See below.
- **Recordings:** All recordings can be found here. (You need your ETH login for web applications (LDAP).)
- **Exam preparation:** There will be two online Q&A sessions and the dates will be published here. (You need your ETH login for web applications (LDAP).)

Summary

This lecture provides an overview of programming techniques for scientific simulations. The focus is on basic and advanced C++ programming techniques and scientific software libraries. Based on an overview over the hardware components of PCs and supercomputer, optimization methods for scientific simulation codes are explained.

A rough weekly roadmap is as follows:

1. Outline; Version control system (with git);
2. Structure of a program (header + source); Preprocessor, compiling, linking; Build system (Make);
3. Build system (CMake); Function templates; Generic programming introduction; Classes;
4. More on classes: Operators, function objects; Lambda expressions;
5. Even more on classes: Class templates; Type traits;
6. Generic programming; Concepts/named requirements; Error handling (exceptions); Testing; Standard library I: Random numbers, Timing; Documentation;
7. Standard library II: Containers, Algorithms, Iterators;

8. Inheritance; Run/Compile-time polymorphism; From modular to generic programming;
9. Introduction to hardware: CPU & memory hierarchy;
10. Compiler optimizations; General optimization; Matrix-matrix multiplication; Blocking; Linear algebra libraries (BLAS, LAPCK, ...);
11. Optimization in C++: Template metaprogramming, lazy evaluation, expression templates;
12. Python introduction I: Basics, classes, modules, packages, I/O, exceptions;
13. Python introduction II: Standard packages for scientific computing (NumPy, SciPy, Matplotlib, h5py); C++ I/O;
14. Summary; Euler bonus;

Questions

Please ask your questions during the lecture/exercise classes or the weekly Q&A session (upon registration & online over Zoom) every Monday 17:00-18:00. Please note that questions relating to **any** lectures/exercises are welcome.

Lecture slides, script, exercises and solutions

Lecture slides, exercise sheets and solutions will be provided as part of this git repository. The lecture script is provided as a git submodule. In order to pull that repository you need to initialize it with the following command:

```
git submodule update --init --remote
```

In order to check and get updates from the script repo, use the following:

```
git submodule update --remote
```

Exercise submission

If you want to receive feedback on your exercises, please push your solutions to your own git repository before **Monday night** of the week after we hand out the exercise sheet. Then create a **GitLab** issue. Please make sure to tag the PT1 team @apietak, @gfederic, @jbachmann, @lbrodbeck and @mkryukov in the issue description, possibly with specific questions.

Your exercise will then be corrected before the next exercise session. Make sure to give *maintainer* access to the PT1 team @apietak, @gfederic, @jbachmann, @karoger, @lbrodbeck and @mkryukov (see here).

Of course, working in small groups is allowed (and even encouraged using a collaborative workflow with **git** and **GitLab**). However, please make sure that you understand every part of the group's proposed solution (you will have to e.g., at the exam!). If several group members submit the exercises, please indicate clearly in the notification/request for correction all the group members and indicate which parts of the solution you would like to have looked at individually

(although we try to run data comparison tools carefully during the correction, we may miss some individual solution of group members).

Course confirmation (Testat)

For doctoral students needing the course attendance confirmation (Testat) for this course, we require that 70% of the exercises have been solved reasonably (sinnvoll). The submission deadline is every Wednesday midnight (Zurich time!).

Please announce that you want the course attendance confirmation (Testat) for this course explicitly at the beginning of the semester by contacting Roger Kaeppli by email. Beforehand, please check with your department's study administration if the course attendance confirmation (Testat) is what you need.

Exam information

- For general information, see the performance assessment tab in the course catalogue [here](#).
- The exam will have two parts: A written theoretical part, and a programming part that you will solve on the exam computers.
- The exam computers will run Ubuntu Desktop Linux. The system language of the computers is English. We only recommend the use of the following simple text editors: GNOME Text Editor (GUI), gedit (GUI), nano (CLI), (neo)vi(m) (CLI/GUI) and (x)emacs (CLI/GUI). Likewise, we only recommend the use of the following terminal emulators: GNOME Terminal and xterm. Although other editors (or even IDEs) and terminal emulators may be available, we will not provide any support. Use at your own risk. We *strongly* recommend that you check out an Ubuntu Desktop system beforehand!
- By default, the keyboards will have the Swiss layout. There will be a poll for those who want to get a US keyboard instead (limited availability: first-come, first-served).
- Provided on the computers are:
 - The full lecture repository (including the script repository; note that only offline versions are provided and links to the web will not work)
 - The C++ standard (draft version)
 - An offline version of <http://www.cppreference.com> (see *Html book* <https://en.cppreference.com/w/Cppreference%253AArchives.html>). We recommend that you try it out before the exam. Also note that the search function is absent: use the Index page and the search function of the browser.
 - An offline version of the Python documentation (see <https://docs.python.org/3/download.html>). We recommend that you try it out before the exam.
 - As needed, offline versions of the documentation for Python libraries.

- Remember: there are manual pages (man, info, help, ...) available!
- This is an open-book exam, which means that you can bring any written material (books, notes, printed code, ...). However, you may **not** use any digital devices (other than the exam computer) during the exam.
- Don't forget to bring your student card (Legi).

Useful resources

Web resources

- C++ reference
- C++ standard
- C++ core guidelines

Debugging

Sometimes it is useful to use a so-called debugger to catch subtle errors. See the following resources:

- GNU Debugger (GDB)
- GNU Data Display Debugger: a graphical user interface for command-line debuggers such as GDB
- A short GDB tutorial

Many editors can interface with GDB directly. Please see their respective documentation.

Literature

C/C++ primers and references:

- Stroustrup, “The C++ Programming Language”, 2013. Available online within the ETH network [here](#).
- Kernighan and Ritchie, “C Programming Language”, 1988.

Practical/Advanced C++ programming:

- Meyers, “Effective STL”, 2001. Available online within the ETH network [here](#).
- Meyers, “Effective C++: 50 Specific Ways to Improve Your Programs and Designs”, 2005. Available online within the ETH network [here](#).
- Meyers, “Effective Modern C++”, 2014. Available online within the ETH network [here](#).
- Vandevorde, Gregor, and Josuttis, “C++ Templates: The Complete Guide”, 2017. Available online within the ETH network [here](#).

Remote Login to (Fedora)

Some Fedora Linux machines (slab[1-4].ethz.ch) are available by remote login with Secure SHell (SSH). See the following website (German):

- [https://cd-portal.sp.ethz.ch/linux/Wiki/Remote%20Login%20\(Fedora\).aspx](https://cd-portal.sp.ethz.ch/linux/Wiki/Remote%20Login%20(Fedora).aspx)
(only accessible within ETH network, e.g., VPN)

For using SSH, please check out the following Working Remotely Tutorial

- <https://carpentries-incubator.github.io/shell-extras/02-ssh/index.html>